Developer Guide — Mobile Application

Welcome. This guide is written for someone who is **new to Flutter** and new to this codebase. It explains the Flutter/Dart concepts you'll bump into as you read the code, then walks the actual project directory, folder by folder, file by file, explaining *why* each thing exists.

For build/run commands, see `DEVELOPMENT.md` at the repo root. This guide is about understanding the code, not running it.

---

1. Flutter concepts you need before reading any file

If you already know Flutter, skip to §2.

Everything is a widget

Flutter has no separate "markup" language — the UI **is** Dart code. Every visible thing (a button, a padding gap, a whole screen) is an object called a **widget**. Widgets nest inside each other to build a screen, e.g. `Scaffold > Column > Text`. You'll see this nesting constantly in this codebase's `build()` methods.

Two widget base classes matter here:

- `StatelessWidget` — has no mutable state of its own; it just renders based on the data passed into it. Example: `lib/core/widgets/premium_text_field.dart`.
- `StatefulWidget` — pairs with a `State` object that can call `setState(() { ... })` to trigger a rebuild when something changes internally (e.g. a press animation). Example: `lib/core/widgets/gradient_button.dart`, which tracks whether the button is currently pressed.

`build(BuildContext context)`

Every widget has a `build` method that returns the widget tree it wants to display. Flutter calls `build` again whenever the widget's state (or a `Provider` it's watching — see below) changes. **`build` methods must be cheap and side-effect-free** — no network calls, no `setState` calls inside `build` itself.

State management: Riverpod

Plain widget state (`setState`) only works for state that's local to one widget. This app needs state that's shared across the whole app — "is the user logged in?", "what did they type in the login form?" — and that's what Riverpod is for.

The pattern used everywhere in this app:

```
class SomeState { ... }                                  // 1. plain data class

class SomeNotifier extends StateNotifier<SomeState> { // 2. holds + mutates the state
  SomeNotifier() : super(const SomeState());
  void doSomething() => state = state.copyWith( ... );  // never mutate in place - always rep
}

final someProvider =                                     // 3. the global handle other code us
    StateNotifierProvider<SomeNotifier, SomeState>((ref) => SomeNotifier());
```

Then, in any widget:

- `ref.watch(someProvider)` — read the current state **and** rebuild this widget whenever it changes. Use this in `build()`.
- `ref.read(someProvider.notifier).doSomething()` — call a method on the notifier without subscribing to changes. Use this in event handlers (`onPressed`, etc.), never in `build()`.
- `ref.listen(someProvider, (previous, next) { ... })` — run a one-off side effect (show a snackbar, navigate) when the state changes, without rebuilding the widget itself.

A widget that needs `ref` extends `ConsumerWidget` (stateless + Riverpod) or `ConsumerStatefulWidget` / `ConsumerState` (stateful + Riverpod) instead of the plain Flutter base classes.

Real example — `lib/features/auth/presentation/providers/auth_provider.dart` defines `AuthState` / `AuthNotifier` / `authProvider`; `login_screen.dart` `ref.watch`es it to show a spinner, and `home_screen.dart` `ref.watch`es it to display the signed-in user's name.

Routing: go_router

Instead of Flutter's default `Navigator.push( ... )`, this app declares its screens as a table of URL-like paths in `lib/core/router/app_router.dart`, and navigates with `context.go('/home')`. This gives us:

- A single place that decides "is this route allowed right now?" (the `redirect` callback — bounces signed-out users to `/login`, bounces signed-in users away from `/login`).
- Deep-linkable, browser-back-button-friendly routes when running on web.

Immutable data + `copyWith`

Notice `AuthState`, `LoginFormState`, `NavItem`, etc. never expose a setter — every field is `final`, and change happens by constructing a *new* instance via `copyWith( ... )`. This is a deliberate Flutter/Riverpod convention: Riverpod detects "did the state change?" by object identity, so mutating a field in place wouldn't trigger a rebuild.

Code generation (`freezed` / `json_serializable`)

`lib/features/auth/data/models/login_response_model.dart` is annotated `@freezed` and has two `part` files (`login_response_model.freezed.dart` / `.g.dart`) that don't exist until you run:

```
dart run build_runner build --delete-conflicting-outputs
```

This generates `fromJson` / `toJson` / `copyWith` / `==` for you so you don't hand-write JSON parsing boilerplate. Re-run this command any time you change a `@freezed` class.

---

2. Project structure

```
MobileApplication/
├── DEVELOPMENT.md              # build/run instructions + architecture decisions
├── docs/
│   ├── DEVELOPER_GUIDE.md      # this file
│   ├── fixes/                  # write-ups of past bugs and how they were fixed
│   └── plans/                  # implementation plans for features, written before buildin
└── app/                        # the actual Flutter project — see below
```

Everything Flutter-specific lives under `app/`, not the repo root, because this repo may eventually hold the backend too (see `DEVELOPMENT.md` §1).

`app/` top level

Path	Purpose
`lib/`	**All the Dart source code you'll actually edit.** Everything else in `app/` is generated or platform plumbing.
`pubspec.yaml`	The Dart/Flutter equivalent of `package.json` — declares dependencies (Riverpod, go_router, Dio, etc.) and the SDK version. Run `flutter pub get` after editing it.
`pubspec.lock`	Exact resolved versions of every dependency (like `package-lock.json`). Don't hand-edit; it's regenerated by `flutter pub get` .
`analysis_options.yaml`	Lint rules (`flutter_lints`) — what `flutter analyze` checks.
`android/` , `ios/` , `windows/` , `web/`	Platform-specific scaffolding generated by `flutter create` . You rarely touch these unless adding a platform permission (e.g. the biometrics entry in `android/app/src/main/AndroidManifest.xml`) or changing app icons/branding.

Path	Purpose
`test/`	Automated tests (`widget_test.dart` today — a smoke test for the login screen).

`lib/` — the app itself

```
lib/
├── main.dart
├── app.dart
├── core/
│   ├── config/
│   ├── network/
│   ├── router/
│   ├── storage/
│   ├── services/
│   ├── theme/
│   └── widgets/
└── features/
    ├── auth/
    ├── home/
    ├── shell/
    ├── shipments/
    ├── notifications/
    └── profile/
```

`lib/main.dart` — the entry point

```
void main() {
  runApp(const ProviderScope(child: MyApp()));
}
```

This is where the program starts (like `main()` in most languages). `ProviderScope` is Riverpod's root widget — it must wrap the entire app or `ref.watch` / `ref.read` won't work anywhere. `MyApp` is defined in `app.dart`.

`lib/app.dart` — the root widget

Builds `MaterialApp.router`, which wires in: - `AppTheme.light` / `AppTheme.dark` (see `core/theme/`) - `ThemeMode.system` — follows the OS light/dark setting - `routerProvider` (see `core/router/`) as the navigation source of truth

`lib/core/` — shared building blocks, used by every feature

Anything in `core/` must **not** import anything from `features/`. The dependency direction only goes one way: features depend on core, never the reverse. This keeps `core/` reusable and stops circular imports.

- `core/config/env.dart` — the backend's base URL, read via `--dart-define=API_BASE_URL=...` at build/run time so the same code works against local/staging/prod backends without hardcoding a URL. Defaults to a local backend for convenience.
- `core/network/`
- `dio_client.dart` — wraps Dio (the HTTP client) with one interceptor that attaches `Authorization: Bearer <token>` to every outgoing request (reading the token from `SecureStorage`), and another that turns raw HTTP/network errors into a friendly `ApiException`.
- `api_exception.dart` — a small `Exception` subclass carrying just a user-presentable `message` (+ optional status code). Every repository catches Dio's raw errors and re-throws this instead, so UI code never has to know about Dio-specific error types.
- `core/router/app_router.dart` — the single table of every route in the app, plus the auth guard (`redirect`). See §1's "Routing" section above for the concept; see §3 below for exactly how this file is structured today.
- `core/storage/secure_storage.dart` — thin wrapper over `flutter_secure_storage` (Android Keystore / iOS Keychain-backed, **not** plain `SharedPreferences`) for the two things that must never sit in plaintext: the auth token and the "remember me" flag.
- `core/services/biometric_service.dart` — wraps the `local_auth` plugin (Face ID / fingerprint). `isSupportedPlatform` is `false` on web/desktop so callers can hide biometric UI there instead of crashing.
- `core/theme/` — everything about how the app *looks*, kept separate from what it *does*:
- `app_colors.dart` — the brand palette (`primary` / `secondary` / `accent`, semantic `success` / `warning` / `error`) plus the two gradients and the `glassFill()` / `glassBorder()` helpers every "glass" surface (`GlassCard`, `PremiumSideMenu`) uses to adapt to light/dark mode.
- `app_text_styles.dart` — builds a `TextTheme` on top of Google Fonts' Poppins, at the specific sizes/weights this app's design calls for.
- `app_theme.dart` — assembles the two above into a full Material 3 `ThemeData` (one for light, one for dark) — button shapes, input field borders, page-transition animation per platform, etc. `app.dart` just plugs these in; individual screens shouldn't need to override theme values inline.
- `core/widgets/` — reusable UI pieces with no feature-specific logic (they take plain data/callbacks in, and know nothing about Riverpod providers or `User` / `AuthState`):
- `animated_gradient_background.dart` — the drifting blurred-orb backdrop behind auth screens.
- `glass_card.dart` — the actual glassmorphism effect (`BackdropFilter` + translucent tint + border), used by `GlassCard` on the login screen.
- `gradient_button.dart` — the primary CTA button: gradient fill, press-scale animation, and an `AnimatedSwitcher` that cross-fades between the label and a loading spinner. **Read** `docs/fixes/login-duplicate-key-crash.md` if you touch this file — there's a subtle gotcha here about gating `onTap` the same way as the other gesture callbacks.

- `premium_text_field.dart` — the floating-label input used for User ID/Password, themed entirely through `AppTheme.inputDecorationTheme` rather than styling itself.
- `social_login_button.dart` / `brand_glyphs.dart` — a reusable glass-pill button shell and placeholder Google/Apple/Microsoft marks for the (currently commented-out, not-yet-wired) social login row.

`lib/features/` — one folder per feature, each split into `data` / `domain` / `presentation`

This is **Clean Architecture, feature-first**: instead of one giant `models/`, one giant `screens/`, etc., each feature is self-contained, and inside each feature the code is layered:

- `domain/` — plain business objects and rules. No JSON, no Riverpod, no Flutter widgets, no networking. This layer shouldn't change just because the backend's response shape changes.
- `data/` — talks to the outside world (backend API, local storage) and maps raw responses onto `domain/` objects. This is the *only* layer that should need to change if the backend's JSON shape changes.
- `presentation/` — Riverpod providers/notifiers plus the actual screens/widgets the user sees. Depends on `domain/` (and sometimes `data/`, via a provider), never the other way around.

If you're adding a new feature (e.g. "profile"), give it the same three folders, even if one starts out nearly empty.

`features/auth/` — login

```
auth/
├── data/
│   ├── models/login_response_model.dart      # raw backend JSON shape
│   └── repositories/auth_repository.dart     # POST /Auth/login, maps response → User, pers
├── domain/
│   └── entities/user.dart                    # plain User{id, name, groupId, empCode, descr
└── presentation/
    ├── providers/
    │   ├── auth_provider.dart                # AuthState/AuthNotifier — isLoading/user/erro
    │   └── login_form_provider.dart          # form field values + validation, independent
    └── screens/
        └── login_screen.dart                 # assembles GlassCard + PremiumTextField + Gra
```

Why `user.id` isn't part of `LoginResponseModel`: the backend's `/Auth/login` response is flat and doesn't return a stable user identifier (see the JSON sample in `DEVELOPMENT.md` §7.2) — so `AuthRepository.login()` threads the user-typed login ID through as `User.id` itself. This is exactly the kind of backend quirk the `data/` layer exists to absorb, so `domain/User` stays clean.

`features/home/` — the post-login dashboard

```
home/
└── presentation/
    └── screens/home_screen.dart   # dashboard body content — no Scaffold/AppBar of its own
```

`HomeScreen` is a `ConsumerWidget` that `ref.watch`es `authProvider` just for the greeting name. It renders a fixed `ListView`: a header (greeting + gradient-avatar initials, via `core/utils/initials.dart`), a 2×2 KPI `GridView` (active shipments / in transit / at customs / delivered today — currently hardcoded `'0'` placeholders, not yet wired to a real summary endpoint), and up to two "active container" cards. The `shipments` list it renders is a constructor parameter, not fetched by the screen itself — see `features/shipments/` below for where that data belongs once a real API exists.

`features/shell/` — the persistent post-login navigation chrome

Added to give every authenticated screen a consistent header/navigation, instead of each screen building its own `Scaffold` / `AppBar` /logout button. See `docs/plans/premium-navigation-side-menu.md` for the design rationale.

```
shell/
└── presentation/
    ├── nav_items.dart                  # NavItem{icon, selectedIcon, label, path} list —
    ├── screens/app_shell.dart          # responsive frame: side rail (≥900px) or bottom
    └── widgets/
        ├── premium_side_menu.dart      # wide-layout glassmorphic side rail: avatar head
        └── bottom_nav_bar.dart         # narrow-layout bottom bar with a centered scan F
```

`AppShell` picks between the two nav widgets at a single breakpoint (`_wideBreakpoint = 900.0`, checked via `MediaQuery.sizeOf(context).width`) rather than using two separate routes — so resizing a window (e.g. on Windows/web) swaps the chrome live without losing navigation state.

How it plugs into routing: `app_router.dart` wraps `/home` (and every other authenticated route) in a `ShellRoute`, whose `builder` returns `AppShell(child: <the matched screen>)`. That means `AppShell` — and therefore the side menu / bottom bar — stays mounted across navigation between authenticated screens, instead of being torn down and rebuilt each time (which would replay its entrance animation and reset nav state on every tap). `/login` is deliberately **outside** the `ShellRoute` since it has its own full-screen layout.

`features/shipments/` — container tracking

```
shipments/
├── domain/
│   └── shipment.dart                       # ShipmentStatus enum + Shipment/JourneyStep e
└── presentation/
    ├── screens/
    │   ├── shipment_list_screen.dart        # searchable, filterable list (mockup 3b)
    │   ├── shipment_detail_screen.dart      # vertical customs/journey timeline (mockup 3
    │   └── scan_screen.dart                 # barcode/Bayan-QR scan UI (mockup 3e)
    └── widgets/
        └── shipment_widgets.dart            # StatusChip + RouteProgress, reused by list/
```

`Shipment` lives in `domain/`, not `data/` — there's no `data/` layer here yet because nothing calls a real backend for shipments. Every screen currently receives its `shipments` list as a constructor parameter supplied by whoever builds it (today: empty lists, wired up in `app_router.dart`). When a shipments API exists, add a `data/repositories/shipment_repository.dart` + a Riverpod provider, mirroring `features/auth/` exactly.

`ShipmentStatus` is a good example of Dart **extension methods**: instead of a `switch` scattered across every widget that needs a status's label, color, or icon, `ShipmentStatusX` (in `shipment.dart`) attaches `.label`, `.color`, and `.icon` getters directly onto the enum, so call sites just write `s.status.color`.

`ScanScreen`'s camera view (`_ScannerFrame`) is a **stubbed placeholder** — an animated gradient box with a sweeping scan-line, not a real camera feed. Wire in a plugin such as `mobile_scanner` when integrating an actual barcode/QR scanner.

`features/notifications/` — alerts feed

```
notifications/
└── presentation/
    └── screens/alerts_screen.dart   # AppNotification model + list UI (mockup 3f)
```

`AppNotification` is defined directly inside `alerts_screen.dart` rather than under a `domain/` folder — like shipments, nothing populates it from a real backend yet. `AlertsScreen` takes a `notifications` list via its constructor, empty by default.

`features/profile/` — signed-in account screen

```
profile/
└── presentation/
    └── screens/profile_screen.dart
```

Reads `authProvider`'s `user`, same as `home_screen.dart` and `premium_side_menu.dart`. This screen is the **only place to log out on the narrow/mobile layout** — `AppBottomNavBar` has no logout

button, so on a phone-sized window `ProfileScreen`'s log-out row is the only way to sign out (on wide layouts, `PremiumSideMenu` also has one).

---

3. Following one request end-to-end: logging in

Reading the layers in isolation only gets you so far — here's the actual call path for a login, file by file, so you can see how the pieces defined above connect:

1. **`login_screen.dart`** — user types into two `PremiumTextField`s. Each keystroke calls `formNotifier.setUserId` / `setPassword` (`login_form_provider.dart`), which updates `LoginFormState` and marks the field "touched" so validation errors can appear.
2. User taps the `GradientButton` → `_submit()` in `login_screen.dart`.
3. `_submit()` checks `form.isValid`; if invalid, it just flips the touched flags (so errors show) and returns — no network call.
4. If valid, it calls `ref.read(authProvider.notifier).login(...)`.
5. **`auth_provider.dart`**'s `AuthNotifier.login()` sets `isLoading: true`, then calls **`auth_repository.dart`**'s `login()`.
6. **`auth_repository.dart`** POSTs to `/Auth/login` via **`dio_client.dart`** (which the request interceptor stamps with any existing auth header — irrelevant pre-login, but the same client is reused for every authenticated request afterward).
7. The raw JSON response is parsed into **`login_response_model.dart`**'s generated `fromJson`. If `success` is false, an `ApiException` is thrown; otherwise `toEntity(userId)` maps it onto a **`domain/User`**.
8. Back in `AuthNotifier`, success sets `isLoading: false, user: <User>`; failure sets `isLoading: false, error: <message>`.
9. That state change does two things simultaneously, because two different widgets are watching/listening to `authProvider`: - `login_screen.dart`'s `ref.listen` fires a snackbar for a new error, or "Welcome back" on success. - `login_screen.dart`'s `ref.watch` flips `GradientButton.isLoading` back to `false`. - **`app_router.dart`**'s `_AuthRefreshNotifier` (listening via `ref.listen` inside its constructor) sees `isAuthenticated` flip from `false` to `true` and calls `notifyListeners()`, which tells `go_router` to re-run its `redirect` callback.
10. `redirect` sees `isAuthenticated && isLoggingIn` and returns `/home`.
11. `/home` is inside the `ShellRoute`, so **`app_shell.dart`** builds around it, and **`home_screen.dart`** renders as its body content, reading the now-populated `user` back out of `authProvider`.

If you're changing login behavior, this is the list of files a change is likely to ripple through.

---

4. A second example: opening a shipment (no Riverpod, no go_router)

Not everything routes through `go_router` or Riverpod — it's worth seeing the simpler pattern too, since most of `features/shipments/` and `features/notifications/` still use it today:

1. `HomeScreen._containerCard` (or `ShipmentListScreen._row`) wraps a `SoftCard` whose `onTap` calls a local `openShipment(Shipment s)` closure defined right inside `build()`.
2. That closure does a **plain, non-go_router** navigation: `Navigator.of(context).push(MaterialPageRoute(builder: (_) => ShipmentDetailScreen(shipment: s)))`. `ShipmentDetailScreen` isn't a route in `app_router.dart` at all — it's pushed directly on top of whichever screen triggered it, and its own back arrow calls `Navigator.of(context).maybePop()` to return.
3. `ShipmentDetailScreen` is a plain `StatelessWidget` — no provider, no `ref`. Every piece of data it renders (the `Shipment`, including its `journey` list of `JourneyStep`s) was handed to it directly as a constructor argument by whichever screen pushed it.

When to reach for this vs. a go_router route + provider: use plain `Navigator.push` for a screen that's always opened *from* another specific screen with the data already in hand (a detail view, a modal-ish flow like `ScanScreen`). Reserve `go_router` routes for top-level, independently reachable destinations that the auth guard or deep links need to know about — every entry in `nav_items.dart` **is** a `go_router` route for exactly this reason.

---

5. Conventions worth knowing before you write new code

- **Comments explain *why*, not *what*.** You'll see very few comments describing what a line does (the code + naming should already make that obvious) and more explaining a non-obvious constraint or backend quirk. Follow that pattern rather than narrating your code.
- `copyWith`**, never in-place mutation**, for any Riverpod state class.
- `core/` **never imports** `features/`**.** If a widget needs a `User` or a feature-specific provider, it belongs under that feature's `presentation/`, not `core/widgets/` — see how `PremiumSideMenu` lives under `features/shell/`, not `core/widgets/`, for exactly this reason.
- **Gate every gesture callback the same way.** `GradientButton`'s `onTapDown` / `onTapUp` / `onTapCancel` were correctly disabled during loading, but `onTap` wasn't — see `docs/fixes/login-duplicate-key-crash.md`. When you add a new interactive widget with an "enabled" concept, make sure *every* callback respects it, not just the ones you tested by hand.
- **New features get the same three-folder split** (`data` / `domain` / `presentation`) as `auth/`, even if `data` / `domain` start out thin.
- **Planning docs before big features.** Non-trivial features get a short plan written to `docs/plans/` before the code — see `premium-navigation-side-menu.md` for the template: goal, structure decision, numbered steps referencing exact files.